

Exam 1 (110 points) Due: Friday, October 28, 2024 11:59 pm
COMPSCI 733: Advanced Algorithms and Designs
Benzon Carlitos Salazar (salazarbc24@uww.edu)

Documentation: (5 points) Type your solutions using Latex (www.overleaf.com or <https://www.latex-project.org/>). You may insert the images of hand-drawn figures. Submit your solutions (pdf is enough) to Canvas.

Problem 1: (15 points)

Prove (i.e., using the definitions), or disprove (i.e., using contradiction or with a counterexample).

(a) $f(n) = O(g(n))$ implies that $g(n) = O(f(n))$.

Answer: We aim to disprove the claim that if $f(n) = O(g(n))$, then $g(n) = O(f(n))$. The definition of Big-O notation is as follows:

$$f(n) = O(g(n)) \iff \exists c > 0, n_0 > 0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0.$$

To disprove the statement, we will provide a **counterexample**.

Proof. (By counterexample.) Let us consider the following functions:

$$f(n) = n \quad \text{and} \quad g(n) = n^2.$$

First, we check if $f(n) = O(g(n))$. By the definition of Big-O, we need to show that there exist constants $c > 0$ and n_0 such that:

$$n \leq c \cdot n^2 \quad \text{for all } n \geq n_0.$$

Dividing both sides by n (assuming $n > 0$):

$$1 \leq c \cdot n.$$

This inequality is true for $n \geq 1$ if we choose $c = 1$. Thus, $f(n) = n$ is indeed $O(n^2)$. Now, we check if $g(n) = O(f(n))$. We need to check if there exist constants $c' > 0$ and n'_0 such that:

$$n^2 \leq c' \cdot n \quad \text{for all } n \geq n'_0.$$

Dividing both sides by n (assuming $n > 0$):

$$n \leq c',$$

which is clearly false for large n because n grows without bound. Therefore, $g(n) = n^2$ is *not* $O(n)$. □

(b) If $f(n) = O(g(n))$ and $g(n) = O(h(n))$ then $f(n) = O(h(n))$.

Answer: We aim to prove this statement using the definition of Big-O notation.

By definition, $f(n) = O(g(n))$ means that there exist some constants $c_1 > 0$ and $n_1 > 0$ such that:

$$|f(n)| \leq c_1 \cdot |g(n)| \quad \text{for all } n \geq n_1.$$

Similarly, $g(n) = O(h(n))$ means that there exist constants $c_2 > 0$ and $n_2 > 0$ such that:

$$|g(n)| \leq c_2 \cdot |h(n)| \quad \text{for all } n \geq n_2.$$

Now, we need to show that $f(n) = O(h(n))$, meaning there exist constants $c_3 > 0$ and $n_3 > 0$ such that:

$$|f(n)| \leq c_3 \cdot |h(n)| \quad \text{for all } n \geq n_3.$$

Proof. Since $|f(n)| \leq c_1 \cdot |g(n)|$ for $n \geq n_1$, and $|g(n)| \leq c_2 \cdot |h(n)|$ for $n \geq n_2$, we can substitute the second inequality into the first:

$$|f(n)| \leq c_1 \cdot |g(n)| \leq c_1 \cdot c_2 \cdot |h(n)|.$$

Thus, we have:

$$|f(n)| \leq c_3 \cdot |h(n)| \quad \text{where } c_3 = c_1 \cdot c_2.$$

Now, let $n_3 = \max(n_1, n_2)$, ensuring both conditions hold for all $n \geq n_3$. Therefore, we have shown that there exist constants $c_3 = c_1 \cdot c_2$ and $n_3 = \max(n_1, n_2)$ such that:

$$|f(n)| \leq c_3 \cdot |h(n)| \quad \text{for all } n \geq n_3.$$

Hence, $f(n) = O(h(n))$. □

(c) $f(n) + g(n) = \Theta(\min(f(n), g(n)))$.

Answer: We disprove the statement by providing a counterexample.

Proof. (By counterexample.) Let $f(n) = n$ and $g(n) = n^2$. In this case, the sum of the two functions is:

$$f(n) + g(n) = n + n^2.$$

The minimum of the two functions is:

$$\min(f(n), g(n)) = \min(n, n^2) = n.$$

Now, we check if $f(n) + g(n) = \Theta(\min(f(n), g(n)))$. This would mean there exist constants $c_1, c_2 > 0$ and an integer n_0 such that for all $n \geq n_0$:

$$c_1 \cdot n \leq n + n^2 \leq c_2 \cdot n.$$

Upper bound: We first check if the upper bound $n + n^2 \leq c_2 \cdot n$ holds for large n . For large n , n^2 dominates n , so $n + n^2$ grows like n^2 , which cannot be bounded by $c_2 \cdot n$ for any constant c_2 . Therefore, the upper bound fails.

Lower bound: The lower bound $n + n^2 \geq c_1 \cdot n$ holds for sufficiently large n , since n^2 is positive and dominates n , ensuring that $n + n^2 \geq c_1 \cdot n$.

Since the upper bound fails, we conclude that $f(n) + g(n) \neq \Theta(\min(f(n), g(n)))$. Therefore, the original statement is false. $\square$

(d) $f(n) = O(g(n))$ implies $2^{f(n)} = O(2^{g(n)})$.

Answer: We disprove the statement by providing a counterexample.

Proof. (By counterexample.) Let $f(n) = n$ and $g(n) = \frac{n}{2}$.

(1.) Check if $f(n) = O(g(n))$.

We need to show that there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$,

$$f(n) \leq c \cdot g(n).$$

Substituting $f(n) = n$ and $g(n) = \frac{n}{2}$, we get:

$$n \leq c \cdot \frac{n}{2},$$

which simplifies to:

$$2 \leq c.$$

This holds for $c = 2$, so $f(n) = O(g(n))$.

(2.) Check if $2^{f(n)} = O(2^{g(n)})$.

We need to check whether there exist constants $c' > 0$ and $n'_0 \geq 0$ such that for all $n \geq n'_0$,

$$2^{f(n)} \leq c' \cdot 2^{g(n)}.$$

Substituting $f(n) = n$ and $g(n) = \frac{n}{2}$, we get:

$$2^n \leq c' \cdot 2^{n/2}.$$

Dividing both sides by $2^{n/2}$, we get:

$$2^{n/2} \leq c'.$$

As $n \rightarrow \infty$, $2^{n/2}$ grows without bound, which means no constant c' can satisfy this inequality for large n . Therefore, $2^n \neq O(2^{n/2})$.

Therefore, the statement $f(n) = O(g(n))$ implies $2^{f(n)} = O(2^{g(n)})$ is false, as shown by the counterexample $f(n) = n$ and $g(n) = \frac{n}{2}$. $\square$

(e) $n = O(n^{0.9999999} \log(n))$.

Answer: The statement $n = O(n^{0.9999999} \log n)$ is false.

Proof. Suppose, for contradiction, that $n = O(n^{0.9999999} \log n)$. This would mean that there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$,

$$n \leq c \cdot n^{0.9999999} \log n.$$

Dividing both sides by $n^{0.9999999}$, we get:

$$n^{0.0000001} \leq c \cdot \log n.$$

Then, consider the growth rates of $n^{0.0000001}$ and $\log n$ as $n \rightarrow \infty$:

$n^{0.0000001}$ grows asymptotically faster than $\log n$, since even a small power of n grows faster than the logarithmic function.

Thus, for sufficiently large n , $n^{0.0000001}$ exceeds $c \cdot \log n$, which contradicts the assumption that $n \leq c \cdot n^{0.9999999} \log n$.

Therefore, the assumption that $n = O(n^{0.9999999} \log n)$ is false. □

Problem 2: (15 points)

Let G be a connected graph with n vertices.

- (a) Prove by induction that G contains a spanning tree.

Theorem. *Let G be a connected graph with n vertices. Then, G contains a spanning tree.*

Proof. We will prove this statement by induction on the number of vertices n in the connected graph G .

Base Case: $n = 1$

If G has only one vertex and no edges, it trivially contains a spanning tree. In this case, the single vertex itself is a spanning tree with no edges, since there are no other vertices to connect. Hence, the graph contains a spanning tree.

Inductive hypothesis: Assume that for any connected graph with k vertices (where $k \geq 1$), there exists a spanning tree. This means that for any connected graph with k vertices, we can find a subgraph that includes all the vertices of G and is a tree (i.e., connected and acyclic).

Inductive step: Now, we consider a connected graph G with $n = k+1$ vertices. We need to show that G contains a spanning tree.

1. Remove a vertex and edge connections:

- Select any vertex v in G and remove it from the graph, along with all edges connected to v .
- The resulting graph G' has k vertices and remains connected (since removing a single vertex from a connected graph cannot disconnect it as long as there are still connections between other vertices).

2. Apply the inductive hypothesis:

- By the inductive hypothesis, the smaller graph G' , with k vertices, contains a spanning tree T' (i.e., a connected, acyclic subgraph that spans all k vertices of G').

3. Reconnect the removed vertex:

- Now, add back the vertex v and one of its edges to any vertex in the spanning tree T' . Since G is connected, such an edge must exist.
- By adding this edge, we ensure that the vertex v is now connected to the spanning tree T' , and since we only added one edge, the resulting graph remains acyclic.

Thus, the graph G with $n = k + 1$ vertices contains a spanning tree.

Conclusion: By induction, we have shown that every connected graph with n vertices contains a spanning tree.

□

- (b) Prove that if G has n or more edges, then G contains a cycle. (Hint: Use Part (a) and a tree of n vertices has $n - 1$ edges).

Theorem. *Let G be a connected graph with n vertices and $m \geq n$ edges. Then, G contains a cycle.*

Proof. We will use the result from Part (a), which states that every connected graph with n vertices contains a spanning tree.

Step 1: **Existence of a spanning tree (from Part (a)).**

By the result from Part (a), we know that G , being a connected graph with n vertices, contains a spanning tree T . A spanning tree is a connected, acyclic subgraph that includes all n vertices of G and has exactly $n - 1$ edges (as every spanning tree of n vertices does).

Step 2: **Analyze the number of edges in G .**

Since G is given to have $m \geq n$ edges, it must have more edges than the spanning tree T , which has exactly $n - 1$ edges.

Step 3: **Adding extra edges creates a cycle.**

The spanning tree T already spans all the vertices of G , meaning it connects all n vertices without any cycles. Now, consider any extra edges in G (there are at least one because $m \geq n$). Each extra edge must connect two vertices that are already connected in the spanning tree T because the tree spans all vertices.

Adding such an extra edge creates a cycle, as it introduces an additional path between two vertices that already have a unique path in the spanning tree.

Since adding any edge to a spanning tree creates a cycle, G , which has n or more edges, must contain at least one cycle. $\square$

Problem 3: (15 points)

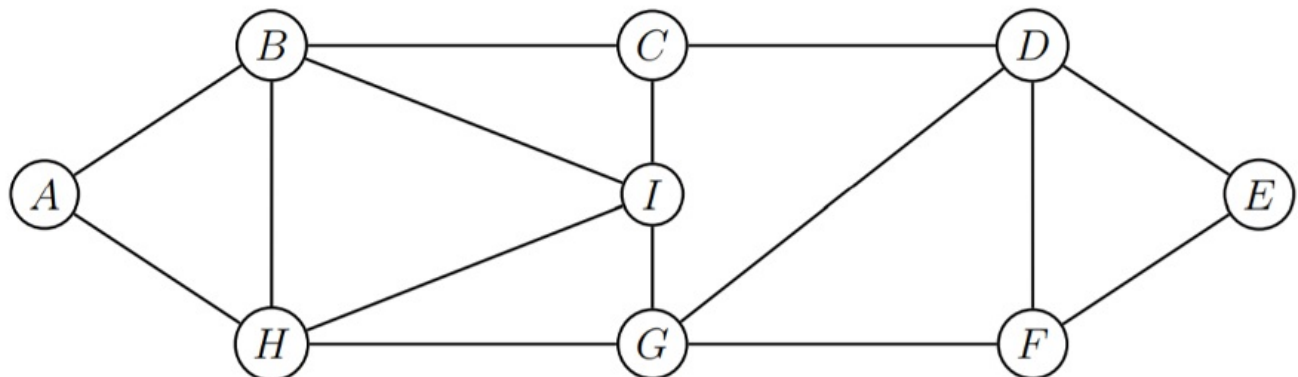
Step through the following DFS algorithm (CLRS textbook) for the following given graph with the starting vertex "A" and fill in the entries in the given table as shown in the class lecture. You need to show any intermediate values too.

Algorithm 1 DFS(G, s)

```
1: for each vertex  $u \in G.V$  do
2:    $u.color \leftarrow \text{WHITE}$ 
3:    $u.\pi \leftarrow \text{NIL}$ 
4: end for
5:  $time \leftarrow 0$ 
6: for each vertex  $u \in G.V$  do
7:   if  $u.color == \text{WHITE}$  then
8:     DFS-VISIT( $G, u$ )
9:   end if
10: end for
```

Algorithm 2 DFS-VISIT(G, u)

```
1:  $time \leftarrow time + 1$ 
2:  $u.d \leftarrow time$ 
3:  $u.color \leftarrow \text{GRAY}$ 
4: for each  $v \in \text{Adj}[u]$  do
5:   if  $v.color == \text{WHITE}$  then
6:      $v.\pi \leftarrow u$ 
7:     DFS-VISIT( $G, v$ )
8:   end if
9: end for
10:  $u.color \leftarrow \text{BLACK}$ 
11:  $time \leftarrow time + 1$ 
12:  $u.f \leftarrow time$ 
```



Intialization	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	WHITE	-	NIL	-
	B	WHITE	-	NIL	-
	C	WHITE	-	NIL	-
	D	WHITE	-	NIL	-
	E	WHITE	-	NIL	-
	F	WHITE	-	NIL	-
	G	WHITE	-	NIL	-
	H	WHITE	-	NIL	-
	I	WHITE	-	NIL	-

Visit A	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	WHITE	-	NIL	-
	C	WHITE	-	NIL	-
	D	WHITE	-	NIL	-
	E	WHITE	-	NIL	-
	F	WHITE	-	NIL	-
	G	WHITE	-	NIL	-
	H	WHITE	-	NIL	-
	I	WHITE	-	NIL	-

Visit B	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	WHITE	-	NIL	-
	D	WHITE	-	NIL	-
	E	WHITE	-	NIL	-
	F	WHITE	-	NIL	-
	G	WHITE	-	NIL	-
	H	WHITE	-	NIL	-
	I	WHITE	-	NIL	-

Visit C	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	GRAY	3	B	-
	D	WHITE	-	NIL	-
	E	WHITE	-	NIL	-
	F	WHITE	-	NIL	-
	G	WHITE	-	NIL	-
	H	WHITE	-	NIL	-
	I	WHITE	-	NIL	-

Visit I	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	GRAY	3	B	-
	D	WHITE	-	NIL	-
	E	WHITE	-	NIL	-
	F	WHITE	-	NIL	-
	G	WHITE	-	NIL	-
	H	WHITE	-	NIL	-
	I	GRAY	4	C	-

Visit D	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	GRAY	3	B	-
	D	GRAY	5	I	-
	E	WHITE	-	NIL	-
	F	WHITE	-	NIL	-
	G	WHITE	-	NIL	-
	H	WHITE	-	NIL	-
	I	GRAY	4	C	-

Visit E	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	GRAY	3	B	-
	D	GRAY	5	I	-
	E	GRAY	6	D	-
	F	WHITE	-	NIL	-
	G	WHITE	-	NIL	-
	H	WHITE	-	NIL	-
	I	GRAY	4	C	-

Finish E	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	GRAY	3	B	-
	D	GRAY	5	I	-
	E	BLACK	6	D	7
	F	WHITE	-	NIL	-
	G	WHITE	-	NIL	-
	H	WHITE	-	NIL	-
	I	GRAY	4	C	-

Finish D

Vertex	Color	Discovery Time	Predecessor	Finishing Time
A	GRAY	1	NIL	-
B	GRAY	2	A	-
C	GRAY	3	B	-
D	BLACK	5	I	8
E	BLACK	6	D	7
F	WHITE	-	NIL	-
G	WHITE	-	NIL	-
H	WHITE	-	NIL	-
I	GRAY	4	C	-

Visit G

Vertex	Color	Discovery Time	Predecessor	Finishing Time
A	GRAY	1	NIL	-
B	GRAY	2	A	-
C	GRAY	3	B	-
D	BLACK	5	I	8
E	BLACK	6	D	7
F	WHITE	-	NIL	-
G	GRAY	9	I	-
H	WHITE	-	NIL	-
I	GRAY	4	C	-

Visit F

Vertex	Color	Discovery Time	Predecessor	Finishing Time
A	GRAY	1	NIL	-
B	GRAY	2	A	-
C	GRAY	3	B	-
D	BLACK	5	I	8
E	BLACK	6	D	7
F	GRAY	10	G	-
G	GRAY	9	I	-
H	WHITE	-	NIL	-
I	GRAY	4	C	-

Finish F

Vertex	Color	Discovery Time	Predecessor	Finishing Time
A	GRAY	1	NIL	-
B	GRAY	2	A	-
C	GRAY	3	B	-
D	BLACK	5	I	8
E	BLACK	6	D	7
F	BLACK	10	G	11
G	GRAY	9	I	-
H	WHITE	-	NIL	-
I	GRAY	4	C	-

Finish G	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	GRAY	3	B	-
	D	BLACK	5	I	8
	E	BLACK	6	D	7
	F	BLACK	10	G	11
	G	BLACK	9	I	12
	H	WHITE	-	NIL	-
	I	GRAY	4	C	-

Finish I	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	GRAY	3	B	-
	D	BLACK	5	I	8
	E	BLACK	6	D	7
	F	BLACK	10	G	11
	G	BLACK	9	I	12
	H	WHITE	-	NIL	-
	I	BLACK	4	C	13

Finish C	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	BLACK	3	B	14
	D	BLACK	5	I	8
	E	BLACK	6	D	7
	F	BLACK	10	G	11
	G	BLACK	9	I	12
	H	WHITE	-	NIL	-
	I	BLACK	4	C	13

Visit H	Vertex	Color	Discovery Time	Predecessor	Finishing Time
	A	GRAY	1	NIL	-
	B	GRAY	2	A	-
	C	BLACK	3	B	14
	D	BLACK	5	I	8
	E	BLACK	6	D	7
	F	BLACK	10	G	11
	G	BLACK	9	I	12
	H	GRAY	15	B	-
	I	BLACK	4	C	13

Finish H

Vertex	Color	Discovery Time	Predecessor	Finishing Time
A	GRAY	1	NIL	-
B	GRAY	2	A	-
C	BLACK	3	B	14
D	BLACK	5	I	8
E	BLACK	6	D	7
F	BLACK	10	G	11
G	BLACK	9	I	12
H	BLACK	15	B	16
I	BLACK	4	C	13

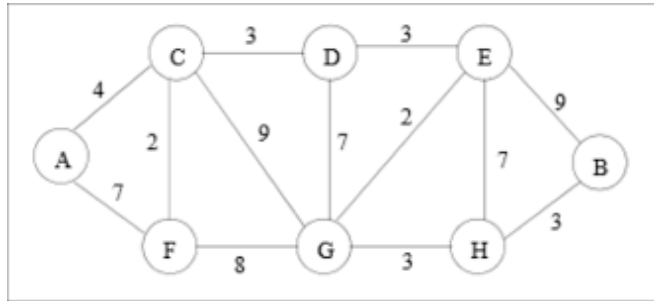
Finish B

Vertex	Color	Discovery Time	Predecessor	Finishing Time
A	GRAY	1	NIL	-
B	BLACK	2	A	17
C	BLACK	3	B	14
D	BLACK	5	I	8
E	BLACK	6	D	7
F	BLACK	10	G	11
G	BLACK	9	I	12
H	BLACK	15	B	16
I	BLACK	4	C	13

Finish A (Final Step)

Vertex	Color	Discovery Time	Predecessor	Finishing Time
A	BLACK	1	NIL	18
B	BLACK	2	A	17
C	BLACK	3	B	14
D	BLACK	5	I	8
E	BLACK	6	D	7
F	BLACK	10	G	11
G	BLACK	9	I	12
H	BLACK	15	B	16
I	BLACK	4	C	13

Problem 4: (15 points) Consider the following graph.



- (a) Use Prim's algorithm to find a minimum-spanning-tree for the above graph. Start from the vertex A. Draw the final MST and give the total weight.

Answer: Prim's Algorithm (Starting from Vertex A)

We start with vertex A and add the minimum weight edge that connects a vertex in the MST to a vertex outside the MST.

1. Start at vertex A.
2. Select the minimum weight edge from A's neighbors: **A – C** with weight 4.
3. The MST now includes vertices {A, C}. The available edges are:
 - A – F with weight 7
 - C – F with weight 2
 - C – D with weight 3
4. Add **C – F** (weight 2) since it's the smallest edge connected to the current MST.
5. The MST now includes vertices {A, C, F}. The available edges are:
 - A – F with weight 7
 - C – D with weight 3
 - F – G with weight 8
6. Add **C – D** (weight 3).
7. The MST now includes vertices {A, C, F, D}. The available edges are:
 - A – F with weight 7
 - F – G with weight 8
 - D – G with weight 9
 - D – E with weight 3
8. Add **D – E** (weight 3).
9. The MST now includes vertices {A, C, F, D, E}. The available edges are:
 - A – F with weight 7
 - F – G with weight 8
 - D – G with weight 9
 - E – H with weight 7

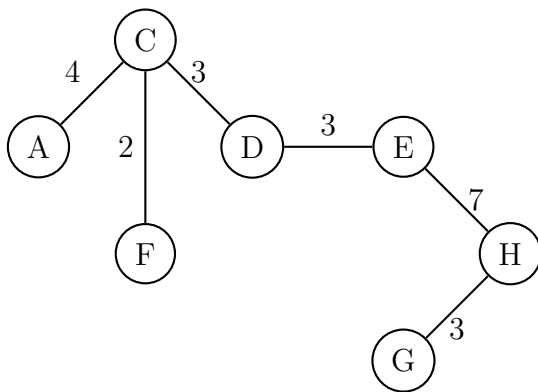
10. Add **E – H** (weight 7).
11. The MST now includes vertices $\{A, C, F, D, E, H\}$. The available edges are:
 - A – F with weight 7
 - F – G with weight 8
 - D – G with weight 9
 - H – G with weight 3
12. Add **H – G** (weight 3).

The edges in the MST are:

- A – C (4)
- C – F (2)
- C – D (3)
- D – E (3)
- E – H (7)
- H – G (3)

Total Weight = $4 + 2 + 3 + 3 + 7 + 3 = 22$.

Minimum Spanning Tree (MST) using Prim's Algorithm



- (b) Use Kruskal's algorithm to find a minimum-spanning-tree for the above graph. Draw the final MST and give the total weight.

Answer: In Kruskal's algorithm, we sort all edges by weight and add edges to the MST as long as they do not form a cycle.

1. Sort edges by weight:
 - C – F (2)
 - C – D (3)
 - D – E (3)
 - H – G (3)
 - A – C (4)

- E – H (7)
- A – F (7)
- F – G (8)
- D – G (9)
- B – E (9)

2. Add edges to the MST in order:

- Add **C – F** (weight 2).
- Add **C – D** (weight 3).
- Add **D – E** (weight 3).
- Add **H – G** (weight 3).
- Add **A – C** (weight 4).
- Add **E – H** (weight 7).

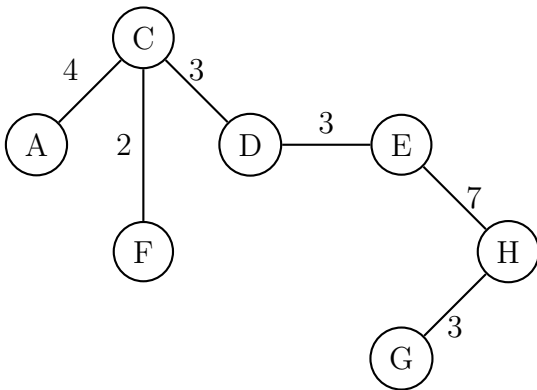
3. Stop here, as we have included all vertices in the MST.

The edges in the MST are:

- C – F (2)
- C – D (3)
- D – E (3)
- H – G (3)
- A – C (4)
- E – H (7)

Total Weight = $2 + 3 + 3 + 3 + 4 + 7 = 22$.

Minimum Spanning Tree (MST) using Kruskal's Algorithm



Problem 5: (15 points)

Let $G = (V, E, W)$ be a weighted connected (undirected) graph, where V is the set of vertices, E is the set of edges, $w_e \in W$ for each $e \in E$ is a nonnegative weight of the edge e . A spanning tree is defined as usual, namely $T = (V, T_E)$ where T_E is a subset of E and T is a tree. The width of a spanning tree T is $\max \{w_e | e \in T_E\}$. Thus it is the most “expensive” edge in the tree. T is a Minimum width spanning tree of G if it is a spanning tree and achieves the minimum width among all spanning trees of G .

- (a) Prove that any minimum spanning tree of G is also a minimum width spanning tree of G . (Hint: You may use proof by contradiction.)

Proof. To prove that any Minimum Spanning Tree (MST) of G is also a Minimum Width Spanning Tree (MWST), we proceed by contradiction.

- Suppose T is a minimum spanning tree of G , but T is not a Minimum Width Spanning Tree.
- This means there exists another spanning tree T' with $\max\{w_e \mid e \in T'_E\} < \max\{w_e \mid e \in T_E\}$, implying that the maximum edge weight in T' is smaller than the maximum edge weight in T .
- However, because T is an MST, it has the smallest possible total weight among all spanning trees in G . The presence of a spanning tree T' with a smaller maximum edge weight suggests that T could be modified to reduce its maximum edge weight without increasing its total weight.
- Specifically, we could replace an edge in T with an edge from T' (if needed) to obtain a tree with the same or lower total weight and a smaller maximum edge weight, which would contradict the assumption that T is an MST.
- Thus, by contradiction, any MST of G must also be a Minimum Width Spanning Tree.

□

- (b) Show with an example that even if the weights of $G = (V, E, W)$ are all distinct, there could be more than one minimum width spanning tree.

Proof. Consider a graph $G = (V, E, W)$ with vertices $V = \{A, B, C, D\}$ and edges with distinct weights as follows:

Edge $A - B$ with weight 1

Edge $A - C$ with weight 2

Edge $B - D$ with weight 4

Edge $C - D$ with weight 3

Edge $B - C$ with weight 5

We can construct two different spanning trees with the same minimum width:

Tree 1: $\{A - B, A - C, B - D\}$ with a maximum edge weight of 4.

Tree 2: $\{A - B, B - D, C - D\}$ with a maximum edge weight of 4.

Both trees are spanning trees that cover all vertices in G and have a maximum edge weight of 4, making them Minimum Width Spanning Trees. Therefore, even with distinct edge weights, multiple Minimum Width Spanning Trees can exist. $\square$

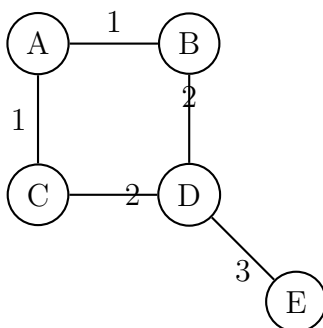
Problem 6: (15 points)

- (a) Draw a weighted graph that has exactly 5 different minimum spanning trees. Explain how you get the count.

Proof. To create a graph with exactly 5 different Minimum Spanning Trees (MSTs), we construct a graph that allows for multiple spanning trees of the same weight. We do this by having several edges with equal weights, providing distinct choices that maintain the total weight for an MST.

1. Consider a graph G with vertices $V = \{A, B, C, D, E\}$.
2. Construct edges as follows:
 - Edge $A - B$ with weight 1
 - Edge $A - C$ with weight 1
 - Edge $B - D$ with weight 2
 - Edge $C - D$ with weight 2
 - Edge $D - E$ with weight 3
3. **Counting MSTs:** We need to select edges to cover all vertices while minimizing the total weight.
 - The edges $A - B$ and $A - C$ each have weight 1. We can choose either of these edges to be in the MST, giving us 2 choices.
 - For edges $B - D$, $C - D$, and $D - E$ with weight 2, we select 2 out of these 3 edges to complete the MST, yielding $\binom{3}{2} = 3$ combinations.
4. Combining these choices, we have $2 \times 3 = 5$ unique MSTs.

Therefore, the resulting graph has exactly 5 different MSTs, all of the same total weight. $\square$



- (b) Draw a weighted graph that has exactly 25 different minimum spanning trees. Explain how you get the count.

Proof. To construct a graph with exactly 25 distinct Minimum Spanning Trees (MSTs), we design a graph where multiple spanning trees have the same total weight by including two sets of edges with five selectable options each.

1. Consider a graph G with vertices $V = \{A, B, C, D, E\}$.
2. Define edges in two weight classes as follows:
 - Five edges of weight 1: $\{A - B, A - C, A - D, A - E, B - C\}$.
 - Five edges of weight 2: $\{B - D, B - E, C - D, C - E, D - E\}$.
3. **Counting the MSTs:** To form each MST, we select one edge from each set of equal-weight edges, ensuring the resulting subgraph is a connected, cycle-free spanning tree with minimum weight:
 - From the five edges of weight 1, choose exactly four to maintain connectivity across all five vertices, resulting in 5 distinct ways to select edges in this class without forming cycles.
 - From the five edges of weight 2, select exactly one to complete the spanning tree, creating a unique MST configuration. There are 5 ways to choose an edge in this class.
4. **Total MST count:** By multiplying the choices, we obtain $5 \times 5 = 25$ unique MSTs, each with the same minimum weight.

Thus, the graph G has exactly 25 different MSTs, all with identical minimum weights. $\square$

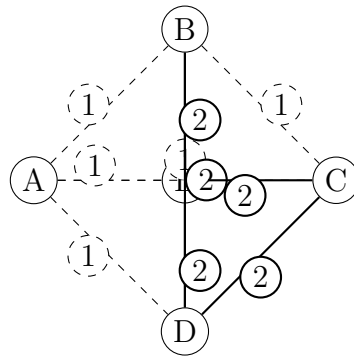


Figure 1: Graph with vertices A, B, C, D, E and two sets of edges: weight 1 (dashed) and weight 2 (solid).

Problem 10: (15 points)

Consider the collection of all undirected graphs with 10 nodes and 6 edges. Let M and m , respectively, be the maximum and minimum number of connected components in any graph in the collection. If a graph has no selfloops and there is at most one edge between any pair of nodes, which of the following is true?

- (A) $M = 10, m = 10$
- (B) $M = 10, m = 1$
- (C) $M = 7, m = 4$
- (D) $M = 6, m = 4$
- (E) $M = 6, m = 3$

Justify your selection by ruling out the other choices. Show two graphs that satisfy M and m .

Answer: We analyze the maximum and minimum possible number of connected components in a graph with 10 nodes and 6 edges. We consider two cases:

- maximizing the number of connected components, and
- minimizing the number of connected components, using only 6 edges.

(1) **Maximum Number of Connected Components (M):**

To maximize the number of connected components, we aim to keep as many nodes isolated as possible. With 10 nodes, we add edges incrementally, ensuring that each edge connects a new pair of nodes to avoid reducing the number of components unnecessarily.

- Adding one edge creates a connected component of 2 nodes, leaving 8 isolated nodes, resulting in 9 connected components. - Adding two edges, each connecting a different pair of nodes, gives two 2-node components and 6 isolated nodes, resulting in 8 connected components. - Adding three edges with each edge connecting a different pair of nodes results in three 2-node components and 4 isolated nodes, which gives 7 connected components. - Adding four edges with each edge connecting a different pair of nodes results in four 2-node components and 2 isolated nodes, which gives 6 connected components.

Thus, the maximum number of connected components achievable with 6 edges is $M = 6$.

(2) **Minimum Number of Connected Components (m):**

To minimize the number of connected components, we want to connect as many nodes as possible into a single component while using only 6 edges.

- Using a tree structure, we can connect up to 7 nodes with 6 edges (since a tree with n nodes requires $n - 1$ edges). - This configuration leaves the remaining 3 nodes isolated.

Therefore, we have one connected component of 7 nodes and 3 isolated nodes, resulting in a minimum of 4 connected components. Hence, $m = 4$.

Conclusion: The correct answer is:

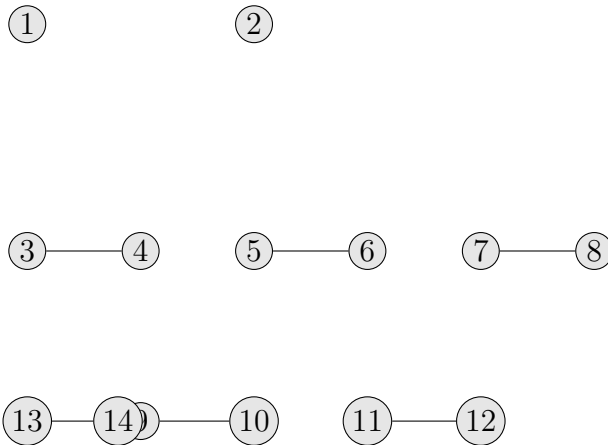
$$(D) \ M = 6, m = 4$$

To rule out the other options:

- (A) $M = 10, m = 10$: This is impossible because, with 6 edges, we cannot have all nodes isolated.
- (B) $M = 10, m = 1$: With only 6 edges, it is impossible to connect all 10 nodes in a single component.
- (C) $M = 7, m = 4$: $M = 7$ is not achievable, as shown by the analysis above.
- (E) $M = 6, m = 3$: $m = 3$ cannot be achieved with only 6 edges, as connecting a large component with 10 nodes would require more edges.

Graph Examples:

1. A graph with $M = 6$ connected components: Six 2-node components and two isolated nodes.



2. A graph with $m = 4$ connected components: One component with 7 nodes (using a tree with 6 edges) and three isolated nodes.

